

Manual técnico — INVENTARIO

Este documento explica, a fondo y en español, cómo está construido el proyecto: qué hace cada carpeta, cada archivo importante y cada decisión de diseño. Está pensado para que puedas leerlo de principio a fin y entender el sistema completo, o usarlo como referencia puntual cuando necesites tocar una parte específica.

No repite lo que ya está en `01-idea.md`, `02-prd.md`, `03-architecture.md` y `api-contract.md` — los complementa bajando al nivel de código real.

1. Visión general

INVENTARIO es una aplicación web de **inventario y facturación** con dos partes independientes que se comunican por HTTP:

- **Backend:** API REST en Node.js + TypeScript + Express, con PostgreSQL como base de datos (a través de Prisma ORM). Vive en `backend/`.
- **Frontend:** aplicación Next.js 16 (App Router) + TypeScript + Tailwind CSS, que consume esa API. Vive en `frontend/`.

Ambas partes corren como procesos separados (por defecto backend en `localhost:4000 / 4010` y frontend en `localhost:3000`) y no comparten código ni memoria — todo lo que "saben" el uno del otro es el contrato HTTP descrito en `docs/api-contract.md`.

Por qué esta separación

Separar backend y frontend en dos aplicaciones (en vez de un monolito Next.js con API routes) da: (a) un backend que se puede desplegar, escalar y proteger de forma independiente; (b) una API reutilizable si mañana quieres una app móvil; (c) capas de seguridad claras (autenticación, autorización, validación) concentradas en un solo lugar, no repartidas entre API routes de Next.js. El costo es tener que mantener un contrato HTTP explícito entre los dos — por eso existe `api-contract.md`.

Los tres roles del sistema

- **ADMIN:** ve todo (incluye la sección "Usuarios" para crear/gestionar trabajadores).
 - **VENDEDOR:** puede operar productos, clientes, facturas y su propio dashboard, pero no ve "Usuarios" ni puede anular facturas.
 - El backend es la única fuente de verdad sobre permisos — el frontend oculta botones/menús por comodidad de uso, pero cada endpoint vuelve a validar el rol de forma independiente. Ocultar un botón en el navegador nunca es suficiente seguridad por sí solo.
-

2. Glosario de términos técnicos usados en este manual

- **API REST:** un backend que expone "recursos" (productos, facturas...) como URLs, y usa los verbos HTTP (`GET` , `POST` , `PATCH` , `DELETE`) para leer/crear/modificar/borrar.
- **ORM (Object-Relational Mapper):** una librería que traduce entre tablas SQL y objetos de tu lenguaje. Aquí es **Prisma**: en vez de escribir `SELECT * FROM "Product"` , escribes `prisma.product.findMany()` y Prisma genera el SQL por ti, con tipos TypeScript incluidos.
- **Migración (migration):** un archivo SQL versionado que describe un cambio en el esquema de la base de datos (agregar una columna, un índice, etc.). Prisma las genera automáticamente comparando `schema.prisma` con el estado real de la base.
- **JWT (JSON Web Token):** un token firmado digitalmente que contiene datos (aquí: `id` , `email` , `role` del usuario) y una firma que prueba que el servidor lo emitió. El cliente lo manda en cada request y el servidor verifica la firma sin necesidad de consultar la base de datos.
- **Middleware:** una función que se ejecuta *antes* del controlador de una ruta, para tareas transversales (verificar autenticación, validar el body, limitar la tasa de requests, etc.).
- **Zod:** librería de validación de esquemas TypeScript. Describe la forma

esperada de un dato (`z.object({ email: z.string().email() })`) y ella lo valida en runtime, cosa que TypeScript por sí solo no puede hacer (los tipos de TS desaparecen al compilar).

- **DTO (Data Transfer Object):** la forma exacta de los datos que viajan entre frontend y backend — no siempre es igual al modelo de base de datos (por ejemplo, nunca se envía `passwordHash`).
 - **Hook (React):** una función que engancha lógica reusable a un componente de React (`useState` , `useEffect` , o los hooks personalizados de este proyecto como `useProducts`).
 - **TanStack Query:** librería que maneja el "estado de servidor" en el frontend — cachea respuestas de la API, evita refetchear de más, maneja loading/error automáticamente, y reintenta si algo fue invalidado.
 - **Zustand:** librería minimalista de estado global de React. Aquí guarda la sesión del usuario (usuario + access token) en memoria.
 - **Componente controlado / no controlado:** un input es "controlado" cuando su valor viene de React state (`value={x} onChange={...}`), y "no controlado" cuando el DOM lleva su propio valor. Mezclar ambos en el mismo campo genera advertencias de React.
 - **Migración de esquema vs. seed:** una migración cambia la *estructura* de las tablas; el seed (`prisma/seed.ts`) llena la base con datos iniciales (el usuario admin, categorías de ejemplo) para poder arrancar a probar.
 - **Rate limiting:** limitar cuántas requests puede hacer un mismo cliente en una ventana de tiempo, para frenar ataques de fuerza bruta o abuso.
 - **CORS (Cross-Origin Resource Sharing):** mecanismo del navegador que bloquea que una página en un dominio llame a una API en otro dominio, salvo que el servidor lo permita explícitamente (aquí, solo se permite el origen configurado en `FRONTEND_URL`).
 - **httpOnly cookie:** una cookie que JavaScript del navegador no puede leer (solo el servidor). Se usa para el refresh token, precisamente para que un script malicioso (XSS) no pueda robarlo.
-

3. Estructura del repositorio

```
INVENTARIO/
  backend/
    src/
      config/          # env.ts, prisma.ts, r2.ts, logger.ts
      middlewares/     # auth, errorHandler, rateLimit, upload, validate
      modules/         # un folder por dominio de negocio
        auth/
        users/
        categories/
        products/
        stock-movements/
        clients/
        invoices/
        dashboard/
      utils/           # AppError, asyncHandler, pagination, serialize
      app.ts           # arma la app Express (middlewares + rutas)
      server.ts        # arranca el servidor HTTP
    prisma/
      schema.prisma    # el esquema completo de la base de datos
      migrations/      # historial de migraciones SQL
      seed.ts          # datos iniciales
    tests/             # tests de integración (Vitest + supertest)
    .env.example
    package.json

  frontend/
    app/
      (auth)/login/    # layout + página de login (grupo de rutas "auth")
      (dashboard)/     # grupo de rutas protegidas
        dashboard/
        productos/
        clientes/
        facturas/
          nueva/
          [id]/
        usuarios/
    components/
      ui/              # primitivos shadcn/base-ui (Button, Select...)
      layout/          # AppShell (sidebar + header)
      auth/, clients/, dashboard/, invoices/, products/, users/, common/
    lib/
      api.ts           # cliente axios + interceptores
      types.ts         # tipos TS que reflejan el contrato de API
      hooks/           # un hook TanStack Query por dominio
      schemas/         # esquemas Zod de formularios
      stores/          # Zustand (auth-store)
    tests/
    .env.example
    package.json
```

```
docs/           # toda la documentación del proyecto (este archivo incluido)
docker-compose.yml # Postgres local para desarrollo
README.md
```

Regla de oro de la organización backend: cada carpeta dentro de

`modules/` es un dominio de negocio autocontenido, y siempre tiene la misma forma:

```
modules/<dominio>/
  <dominio>.routes.ts      # define los endpoints y qué middlewares aplican
  <dominio>.controller.ts  # lee el request, llama al service, arma el response
  <dominio>.service.ts     # la lógica de negocio real (lo único que toca Prisma)
  <dominio>.schemas.ts     # validación Zod de body/query/params
```

El flujo de una request siempre es:

```
Request → routes (aplica middlewares) → validate (Zod) → controller → service → Prisma → P
```

Esto significa: **el controller nunca decide reglas de negocio**, y **el service nunca sabe nada de Express** (no recibe `req / res`, solo recibe datos ya validados y devuelve datos). Esta separación es la que permite, por ejemplo, testear `invoices.service.ts` con datos de prueba sin tener que levantar un servidor HTTP.

4. Backend a fondo

4.1 Arranque (`server.ts` → `app.ts`)

`server.ts` es el punto de entrada: importa `createApp()` desde `app.ts` y llama a `app.listen(env.PORT, ...)`. Se separan a propósito para que los tests puedan importar `createApp()` y probar la app **sin** levantar un puerto real (`tests/helpers/app.ts` hace exactamente eso con `supertest`). `app.ts` (`createApp()`) arma la aplicación Express en este orden exacto — el orden de `app.use(...)` importa, porque cada middleware solo afecta a lo que viene *después* de él:

1. `pino-http` : logea cada request entrante (método, ruta, status, tiempo),

con un `x-request-id` generado o reutilizado del header, para poder rastrear una request específica en los logs.

1. `helmet()` : agrega headers de seguridad HTTP estándar (evita sniffing de MIME type, desactiva `X-Powered-By` , Content-Security-Policy básica, etc.).

1. `cors({ origin: env.FRONTEND_URL, credentials: true })` : solo el frontend configurado puede llamar a la API desde un navegador, y se permiten cookies cross-origin (necesario para el refresh token).

1. `express.json()` : parsea el body de las requests como JSON.
2. `cookieParser()` : permite leer `req.cookies` (se usa para el refresh token).

1. `/health` : endpoint sin autenticación para chequeos de disponibilidad (útil si algún día se despliega detrás de un load balancer).

1. `apiRateLimiter` sobre **todo** `/api/v1` : techo general de 600 requests/15min por IP.

1. Los 7 routers de módulos, cada uno montado en su prefijo (`/api/v1/auth` , `/api/v1/products` , etc.).

1. `notFoundHandler` : captura cualquier ruta no reconocida y responde 404 con el formato de error estándar.

1. `errorHandler` : **siempre al final** — Express reconoce que una función con 4 argumentos (`err, req, res, next`) es un manejador de errores, y solo se ejecuta cuando algo llama a `next(error)` en cualquier punto anterior de la cadena.

4.2 Configuración (`config/`)

- `env.ts` : carga variables de entorno desde `.env` (o `.env.test` cuando `NODE_ENV=test`) con `dotenv` , y las valida con un `z.object({...})` — si falta una variable requerida (por ejemplo `JWT_ACCESS_SECRET`), la app **no arranca** y muestra exactamente qué falta, en vez de fallar de forma confusa más adelante. Las variables `R2_*` (Cloudflare R2, para imágenes de producto) son opcionales — si faltan, la app arranca igual y solo el endpoint de subida de imágenes responde un error controlado.

Detalle importante: usa `dotenv.config({ override: true })` porque Vitest/Vite precargan `.env` de la raíz del proyecto en `process.env` antes de que este archivo se ejecute; sin `override: true`, los tests heredarían silenciosamente la `DATABASE_URL` de desarrollo en vez de la de test.

- `prisma.ts`: crea y exporta una única instancia de `PrismaClient` (patrón singleton) para que toda la app comparta el mismo pool de conexiones a Postgres.
- `r2.ts`: crea el cliente S3 (`@aws-sdk/client-s3`) apuntando al endpoint de Cloudflare R2, solo si las 5 variables `R2_*` están presentes (`isR2Configured`). Si no, `r2Client` es `null` y el resto de la app lo chequea antes de usarlo.
- `logger.ts`: instancia de `pino`, un logger JSON estructurado (rápido, y fácil de ingerir por herramientas de observabilidad si algún día se agregan).

4.3 Middlewares (`middlewares/`)

- `auth.ts`:
- `requireAuth`: lee el header `Authorization: Bearer <token>`, verifica la firma del JWT con `jwt.verify(token, env.JWT_ACCESS_SECRET)`, y si es válido, adjunta el payload decodificado (`{sub, email, role}`) a `req.user`. Si el header falta o el token es inválido/expiró, responde 401 antes de llegar al controller.
- `requireRole(...roles)`: middleware "factory" — se usa como `requireRole("ADMIN")` en la definición de rutas, y devuelve 403 si `req.user.role` no está en la lista permitida. Siempre va *después* de `requireAuth` (necesita que `req.user` ya exista).
- `validate.ts`: middleware genérico `validate({body, query, params})` que recibe hasta 3 esquemas Zod opcionales, los corre contra las partes correspondientes del request, y si algo no matchea lanza un `AppError.badRequest(..., "VALIDATION_ERROR", detallesDeZod)`. Como reemplaza `req.body` / `req.query` / `req.params` con el resultado ya

parseado de Zod, los controllers reciben datos con los tipos y defaults correctos (por ejemplo, `pageSize` ya convertido de string a number).

- `upload.ts` : envuelve `multer` (maneja `multipart/form-data` , guarda el archivo en memoria como `Buffer` , no en disco) con límite de 5MB y una lista blanca de MIME types declarados por el cliente (`image/jpeg|png|webp`). Traduce los errores de multer (archivo muy grande, tipo no permitido, sin archivo) a `AppError` con el formato estándar del resto de la API.

- `rateLimit.ts` : dos limitadores separados con `express-rate-limit` :
 - `loginRateLimiter` : 5 intentos / 15 min, **solo** en `POST /auth/login` — protege contra fuerza bruta de contraseñas.

- `apiRateLimiter` : 600 requests / 15 min, sobre toda la API — un techo general contra un cliente descontrolado o una cuenta comprometida.

Ambos se desactivan automáticamente en tests (`skip: () => isTest`) para no interferir con suites que hacen login muchas veces por diseño. Limitación conocida: el store de límites vive en memoria del proceso — si el backend corriera en más de una instancia, cada instancia tendría su propio contador y el límite real efectivo sería `600 × número de instancias` ; para que sea estricto en ese escenario haría falta un store compartido (Redis).

- `errorHandler.ts` : el único lugar del backend que decide el formato final de una respuesta de error. Distingue:

- `AppError` (errores de negocio que nosotros lanzamos a propósito) → usa su `statusCode` / `code` / `message` / `details` .

- `ZodError` (por si algo se valida fuera del middleware `validate`) → 400 con el mismo formato `VALIDATION_ERROR` .

- Errores conocidos de Prisma: `P2002` (violación de restricción única) → 409 `DUPLICATE_ENTRY` ; `P2025` (registro no encontrado en una operación que lo esperaba) → 404 `NOT_FOUND` .

- Cualquier otra cosa → 500 `INTERNAL_ERROR` genérico (nunca se filtra el mensaje interno real al cliente, solo se loguea con `logger.error`).

Todas las respuestas de error tienen la forma

```
{ error: { code, message, details? } } — es el contrato que el frontend
```

espera en `getApiErrorMessage()`.

4.4 Utilidades (`utils/`)

- **`AppError.ts`** : una clase de error con `statusCode` , `code` (string identificable, ej. `"INSUFFICIENT_STOCK"`) y `message` , más métodos estáticos de conveniencia (`AppError.notFound(...)` , `AppError.conflict(...)` , etc.) para no repetir `new AppError(404, ...)` en todos los services.
- **`asyncHandler.ts`** : envuelve un controller `async` para que si lanza una excepción (por ejemplo, un `await` rechazado), Express la capture y la mande al `errorHandler` automáticamente — sin este wrapper, una promesa rechazada dentro de una ruta Express clásica se perdería silenciosamente.
- **`pagination.ts`** : `getPaginationArgs({page, pageSize})` calcula `skip / take` para Prisma a partir de página/tamaño, con límites de seguridad (`page` mínimo 1, `pageSize` acotado a máximo 100 aunque algo se cuele sin pasar por Zod). `buildPaginatedResponse(data, total, page, pageSize)` **arma la envoltura estándar** `{data, page, pageSize, total}` que usan todos los listados.
- **`serialize.ts`** : recorre recursivamente un objeto/array y (a) convierte `Prisma.Decimal` a string con 2 decimales (para que los montos viajen exactos por HTTP sin errores de punto flotante), y (b) elimina campos sensibles (`passwordHash` , `tokenHash`) por si algún día un service se olvida de excluirlos con un `select` . En la práctica, la mayoría de los services ya usan `select / omit` explícito, pero `serialize` es la última red de seguridad.

4.5 Autenticación y sesión (`modules/auth/`)

Este es el módulo más sensible del sistema, así que vale la pena explicarlo paso a paso.

El modelo de tokens:

- **Access token**: un JWT firmado, vive **15 minutos**, viaja en el header `Authorization: Bearer <token>` de cada request, y el backend lo valida

sin tocar la base de datos (la firma ya prueba que es legítimo). Como dura poco, si se filtrara, la ventana de daño es corta.

- **Refresh token:** un UUID aleatorio opaco (no es un JWT, no lleva información codificada), vive **7 días**, y se guarda en una cookie

`httpOnly, secure, sameSite=strict` — JavaScript del navegador *no puede leerlo* (protección contra XSS). En la base de datos nunca se guarda el valor real, sino su hash HMAC-SHA256 (`hashRefreshToken`), igual que una contraseña: si alguien accediera a la base de datos no podría reconstruir tokens válidos a partir del hash.

`auth.service.ts` :

- `login(input)` : busca el usuario por email, verifica que esté `active` , compara la contraseña con `bcrypt.compare` contra el hash guardado. Si todo es válido, firma un access token y genera + guarda (hasheado) un nuevo refresh token en la tabla `RefreshToken` . Devuelve el usuario "safe" (sin `passwordHash`), el access token, y el refresh token crudo (para que el controller lo ponga en la cookie — el service no sabe nada de cookies/Express).
- `refresh(rawToken)` : busca el refresh token por su hash, valida que exista, no esté revocado, no haya expirado, y que el usuario dueño siga activo. Si todo pasa, hace **rotación**: revoca el token usado y crea uno nuevo, todo dentro de una única `prisma.$transaction` . Esto significa que cada refresh token es de un solo uso — si alguien roba uno y lo usa antes que el usuario legítimo, el siguiente intento del usuario legítimo fallará (señal de que hubo un robo), en vez de que ambos sigan usando el mismo token indefinidamente.
- `logout(rawToken)` : marca el refresh token actual como `revoked` . No borra el registro (queda como historial/auditoría).
- `getMe(userId)` : devuelve los datos públicos del usuario autenticado (usado al cargar la app para saber quién es la sesión activa).

Rutas (`auth.routes.ts`):

```
POST /auth/login      (rate-limited, valida body con loginSchema)
POST /auth/refresh    (lee la cookie, no requiere access token — es justo
                        para cuando el access token ya expiró)
POST /auth/logout
GET /auth/me          (requiere access token válido)
```

4.6 Módulo `users` (gestión de trabajadores, solo ADMIN)

- **Schemas:** `createUserSchema` (name, email, password, role), `updateUserSchema` (name/role/active, todos opcionales — el email *nunca* se puede cambiar por este endpoint, una decisión deliberada para no complicar la relación email↔identidad de login).
- **Service:** `createUser` hashlea la contraseña con `bcrypt` costo 12 (2^{12} iteraciones — un balance estándar entre seguridad y tiempo de cómputo) y rechaza emails duplicados con 409. `updateUser` permite desactivar un usuario (`active: false`), lo cual bloquea futuros logins (chequeo en `auth.service.login`) sin borrar su historial de facturas o movimientos de stock (borrar el registro rompería esas relaciones).
- **Rutas:** todas protegidas con `requireAuth` + `requireRole("ADMIN")` — un VENDEDOR que intente llamar cualquier endpoint de `/users` recibe 403, sin importar lo que el frontend le muestre o no le muestre.

4.7 Módulo `categories`

El más simple del sistema: CRUD básico de categorías de producto (`id`, `name` único). Un producto puede no tener categoría (`categoryId` opcional, `onDelete` implícito de Prisma deja el producto sin categoría si se borra una — en la práctica el frontend no expone borrar categorías con productos asociados desde la UI de un vendedor, pero el modelo lo permite).

4.8 Módulo `products` (y las imágenes)

- **Listado** (`listProducts`): soporta búsqueda por nombre/SKU (`contains`, `mode: insensitive`), filtro por categoría, y un filtro `lowStock` interesante: Prisma no permite comparar dos columnas de la misma tabla directamente en su lenguaje de queries (`stock <= minStock`),

así que se resuelve con una *raw query* SQL de apoyo

(`prisma.$queryRaw`) que trae solo los IDs, y luego un `findMany` normal con esos IDs.

- **Crear/actualizar:** valida que el SKU no esté duplicado antes de escribir (evita depender solo del error 409 de Postgres, aunque ese también existe como red de seguridad vía `errorHandler`).
- **Borrar:** en realidad es un *soft delete* — pone `active: false` en vez de un `DELETE` real. Esto es intencional: un producto ya vendido tiene `InvoiceItem` s que lo referencian, y borrarlo de verdad rompería el historial de facturas pasadas.

Imágenes de producto — el flujo completo de seguridad:

1. El navegador nunca habla directo con Cloudflare R2 (el bucket de almacenamiento). Sube el archivo por `multipart/form-data` a `POST /products/:id/image` , contra **nuestro propio backend**.

1. `middlewares/upload.ts` (multer) hace el primer filtro: tamaño máximo 5MB, y el MIME type que el navegador *dice* que tiene el archivo debe estar en la lista blanca.

1. **Ese primer filtro es falsificable:** el `Content-Type` de un archivo en un multipart lo declara el cliente, así que alguien podría renombrar un archivo malicioso y declarar `Content-Type: image/png` sin que realmente sea un PNG. Por eso, antes de subir nada a R2, `products.service.ts` corre `assertValidImageContent(file)` : lee los primeros bytes del archivo y los compara contra la *firma binaria* real de cada formato (el "magic number"):

- PNG: los primeros 8 bytes son siempre `89 50 4E 47 0D 0A 1A 0A` .
- JPEG: los primeros 3 bytes son `FF D8 FF` .
- WEBP: bytes 0-3 son `RIFF` (ASCII) y bytes 8-11 son `WEBP` (ASCII).

Si el contenido no matchea la firma del tipo declarado, se rechaza con 400 `INVALID_FILE_CONTENT` — sin llegar nunca a Cloudflare ni a la base de datos. Esta validación se ejecuta **primero**, antes de chequear si R2 está configurado, porque es una regla de entrada pura (no depende de

infraestructura externa).

1. Si el contenido es válido y R2 está configurado, el backend sube el buffer a R2 con `@aws-sdk/client-s3 (PutObjectCommand)`, bajo una key única `products/<id>-<uuid>.<extensión>`, y si el producto ya tenía una imagen anterior, borra la vieja de R2 (`DeleteObjectCommand`, ignorando el error si ya no existe — no debe bloquear que se guarde la nueva).

1. Solo se guarda la **URL pública** resultante en `Product.imageUrl` (columna `String?` en Postgres) — el binario de la imagen nunca toca la base de datos.

1. `deleteProductImage` hace lo simétrico: borra el objeto de R2 (si existe configuración) y limpia `imageUrl` a `null`.

Por qué Cloudflare R2 y no guardar la imagen en Postgres o en disco:

guardar binarios en Postgres infla la base de datos y hace los backups lentos; guardar en el disco del servidor no escala si algún día hay más de una instancia del backend corriendo. R2 es compatible con la API de S3 (por eso se usa el SDK `@aws-sdk/client-s3` apuntando a un endpoint distinto), y su capa gratuita (10GB/mes, sin costo por descargas) es suficiente para este proyecto. Las variables `R2_*` son opcionales a propósito: si no están configuradas, el resto de la aplicación sigue funcionando normal — solo el endpoint de imagen responde `400 IMAGE_STORAGE_NOT_CONFIGURED` en vez de tumbar el arranque del backend.

4.9 Módulo `stock-movements`

Cada cambio de stock (venta, anulación, ajuste manual) genera un registro `StockMovement (type: ENTRADA | SALIDA | AJUSTE, quantity, reason` opcional, y `userId` de quien lo generó). Es el historial/bitácora de inventario: permite responder "¿por qué este producto tiene 12 unidades menos que la semana pasada?" sin adivinar. El endpoint de ajuste manual (`POST /products/:id/movements`) lo usa la UI del formulario "Ajustar stock".

4.10 Módulo `clients`

CRUD estándar de clientes (`name` , `nit` opcional pero único cuando se da, `email` , `phone` , `address`). El campo se llama `nit` (Número de Identificación Tributaria, el identificador fiscal en Guatemala) — se renombró desde `documentId` genérico durante el desarrollo para reflejar el dominio real del negocio.

4.11 Módulo `invoices` (el corazón del sistema)

Este módulo tiene la lógica de negocio más delicada porque toca dinero y stock a la vez, así que casi todo pasa dentro de transacciones Prisma (`prisma.$transaction`) — o se aplican **todos** los cambios, o ninguno.

Crear factura (`createInvoice`):

1. Consolida cantidades si el mismo producto aparece más de una vez en el array de items (para no descontar stock dos veces de forma incorrecta si el frontend mandara duplicados).

1. Busca todos los productos involucrados en una sola query.
2. Verifica, por cada línea, que el producto exista, esté activo, y tenga stock suficiente. Si algo no existe → 400 `PRODUCT_NOT_FOUND` . Si hay stock insuficiente en una o más líneas → 409 `INSUFFICIENT_STOCK` , con `details` listando exactamente qué producto y cuánto había disponible vs. cuánto se pidió (así el frontend puede mostrar un mensaje preciso, no solo "algo salió mal").

1. Si todo pasa, calcula `subtotal` (suma de `precio × cantidad` por línea, usando `Prisma.Decimal` para evitar errores de redondeo con `number`), `tax = subtotal × TAX_RATE` (12% por defecto, configurable por variable de entorno), y `total = subtotal + tax` .

1. Crea la `Invoice` + sus `InvoiceItem[]` en una sola operación anidada de Prisma.

1. Por cada producto, decrementa `Product.stock` y crea un `StockMovement` tipo `SALIDA` con motivo `"Venta - Factura #N"` .

1. Todo el paso 1-6 vive dentro de un único `prisma.$transaction(async (tx)`

=> {...}): si el paso 6 fallara a mitad de camino (por ejemplo, un error de conexión), Postgres revierte automáticamente todo lo anterior — nunca queda una factura creada sin su descuento de stock correspondiente, ni viceversa.

Anular factura (`cancelInvoice(id, cancelledByUserId)`):

1. Busca la factura; si no existe → 404. Si ya está **ANULADA** → 409

INVOICE_ALREADY_CANCELLED (no se puede anular dos veces).

1. Por cada línea de la factura, repone el stock (`increment`) y crea un

StockMovement tipo **ENTRADA** con motivo **"Anulación factura #N"** .

El `userId` de ese movimiento es **quien anula** (`cancelledByUserId`), no quien emitió la factura originalmente — son personas potencialmente distintas (un vendedor emite, un admin anula), y el registro de auditoría debe reflejar quién ejecutó cada acción real, no reasignarla al creador original.

1. Marca la factura como `status: ANULADA` , y guarda `cancelledAt` (fecha) y `cancelledByUserId` — este rastro de auditoría se agregó específicamente para poder responder "¿quién anuló esta factura y cuándo?" sin tener que inferirlo de los **StockMovement** .

1. Solo un **ADMIN** puede llegar a este endpoint (`requireRole("ADMIN")`) en `invoices.routes.ts`) — un vendedor recibe 403 antes de que el controller se ejecute.

Listado con scoping por rol: un **VENDEDOR** que pide `GET /invoices` solo ve **sus propias** facturas (`where.userId = requester.id` , agregado automáticamente en el service, el cliente no puede pedir ver las de otro vendedor cambiando un parámetro). Un **ADMIN** ve todas. Esta misma regla de scoping se repite en `dashboard.service.ts` para que las métricas del dashboard de un vendedor reflejen solo su propia actividad.

PDF de factura (`invoices.pdf.ts`): usa `pdfkit` para generar un PDF directamente en el stream de respuesta HTTP (`doc.pipe(res)`), sin guardar ningún archivo temporal en disco. Dibuja el encabezado, datos del cliente, vendedor, una tabla simple de líneas (con coordenadas x/y fijas — pdfkit no tiene un sistema de tablas de alto nivel) y los totales. El frontend lo

descarga vía `downloadInvoicePdf()` en `lib/hooks/use-invoices.ts`.

4.12 Módulo `dashboard`

Expone dos endpoints: un resumen de KPIs (`getSummary`) y un reporte de ventas por rango de fechas (`getSalesReport`, exportable a CSV con `toCsv()`). Ambos respetan el mismo scoping por rol que `invoices` (`userScopeWhere`).

El bug de zona horaria que se corrigió: `new Date("2026-09-17")`

interpreta esa fecha como **medianoche UTC**, no medianoche en la zona horaria del servidor. Guatemala está en UTC-6, así que medianoche UTC del 17 son en realidad las 6pm del día 16 hora local — cualquier venta hecha entre las 6pm y medianoche local del día 16 quedaba fuera del rango "día 16" al filtrar así, y aparecía mezclada con el día siguiente. La corrección fue `startOfLocalDay(dateStr, addDays = 0)`: parsea el string `"YYYY-MM-DD"` manualmente (`año, mes, día`) y construye el `Date` con el constructor de 3 argumentos (`new Date(year, month-1, day)`), que **sí** usa la zona horaria local del proceso Node — la misma que usan `startOfToday()` / `startOfMonth()` en el resto del archivo, para que todo el módulo sea consistente. Además, el filtro "hasta" (`to`) se volvió un límite exclusivo al día *siguiente* (`startOfLocalDay(to, 1)`, comparado con `lt` en vez de `lte`), para incluir el día completo hasta las 23:59:59.999 sin tener que calcular esa hora exacta.

5. La base de datos (`prisma/schema.prisma`) — modelo por modelo

- **User**: `id` (uuid), `name`, `email` (único), `passwordHash` (nunca se expone), `role` (`ADMIN` / `VENDEDOR`), `active`, `createdAt`. Relaciones: `invoices` (facturas que emitió), `cancelledInvoices` (facturas que anuló — relación nombrada `"InvoiceCancelledBy"` porque Prisma necesita distinguir dos relaciones distintas hacia el mismo modelo `Invoice`), `movements` (movimientos de stock que registró), `refreshTokens`.

- **RefreshToken** : `tokenHash` (nunca el token crudo), `expiresAt`, `revoked`. Índices en `userId` y `tokenHash` (las dos formas en que se busca este registro: "todos los tokens de este usuario" y "el token con este hash").
- **Category** : solo `id` y `name` único.
- **Product** : `sku` único, `imageUrl` opcional, `categoryId` opcional (`Category?`), `price` y `cost` como `Decimal(12,2)` (12 dígitos totales, 2 decimales — nunca `Float`, porque los flotantes binarios no representan exacto valores decimales como `0.10`, lo cual en dinero es inaceptable), `stock` / `minStock` como enteros, `active` para el soft-delete. Índices en `name` (búsquedas) y `categoryId` (filtrar por categoría, y para que el `JOIN` implícito de Prisma al incluir `category` sea eficiente).
- **StockMovement** : bitácora de cambios de stock. Índice compuesto [`productId`, `createdAt`] — pensado para la consulta típica "historial de este producto, ordenado por fecha".
- **Client** : `nit` opcional pero único cuando se provee (`String?` @unique — `Postgres` permite múltiples `NULL` en una columna única, así que varios clientes sin NIT no chocan entre sí).
- **Invoice** : `number` autoincremental y único (el número de factura visible al usuario, separado del `id` uuid interno). `status` (`EMITIDA` / `ANULADA`). Los tres montos como `Decimal(12,2)`. Los campos de auditoría de anulación (`cancelledAt`, `cancelledByUserId`, relación `cancelledBy`) son nullable porque la mayoría de las facturas nunca se anulan. Índices en `createdAt` (listados ordenados/filtrados por fecha), `clientId`, `userId` y `cancelledByUserId` (las 4 foreign keys que se usan en filtros o joins frecuentes — sin índice, Postgres tendría que escanear la tabla completa en cada uno de esos filtros a medida que crece).
- **InvoiceItem** : la línea de detalle de una factura, con el precio y subtotal *congelados* al momento de la venta (`unitPrice`, `subtotal`) — deliberadamente no se recalculan desde `Product.price` al leer una factura vieja, porque el precio del producto puede cambiar después y la factura debe reflejar el precio real que se cobró ese día. Índices en

`invoiceId` y `productId`.

Por qué se agregaron los índices de foreign key: Postgres no crea un índice automáticamente sobre las columnas `@relation` — solo sobre la llave primaria. Sin índice explícito, un filtro como

`WHERE "clientId" = '...'` sobre una tabla de facturas grande degrada a un *sequential scan* (revisar fila por fila). Se agregaron durante una revisión de escalabilidad, junto con el límite de `pageSize ≤ 100` en todos los listados paginados (evita que alguien pida `pageSize=999999` y fuerce al backend a traer la tabla completa en una sola respuesta).

Migraciones relevantes de este proyecto (en `prisma/migrations/`, en orden cronológico y aplicadas tanto a la base de desarrollo como a la de test): creación del esquema base, renombrar `documentId` → `nit` en `Client`, agregar `Product.imageUrl`, agregar los índices de foreign key faltantes, y agregar los campos de auditoría de anulación de factura.

`prisma/seed.ts`: crea el usuario admin inicial y categorías de ejemplo. Tiene un guard explícito:

```
`if (NODE_ENV === "production" && adminPassword === DEFAULT_ADMIN_PASSWORD)
{ console.error(...); process.exit(1); }`
```

 — evita que alguien corra el seed en un entorno productivo real dejando la contraseña de admin en el valor por defecto documentado públicamente en este mismo repositorio.

6. Frontend a fondo

6.1 App Router y grupos de rutas

Next.js App Router usa carpetas para definir rutas. Dos "grupos de rutas" (carpetas entre paréntesis, que **no** aparecen en la URL) organizan la app:

- `app/(auth)/`: contiene `layout.tsx` (el fondo split-screen) y `login/page.tsx`. No pasa por `AuthGuard` — es la única parte pública.
- `app/(dashboard)/`: contiene `layout.tsx` (envuelve todo en `<AuthGuard><AppShell>{children}</AppShell></AuthGuard>`) y todas las páginas protegidas: `dashboard/`, `productos/`, `clientes/`, `facturas/`

(+ `facturas/nueva/` y `facturas/[id]/` dinámica), `usuarios/` .

- `app/page.tsx` : la raíz `/` no renderiza nada — solo hace `router.replace("/dashboard")` en un `useEffect` , delegando a `AuthGuard` la decisión real de si mostrar el dashboard o mandar a `/login` .

6.2 Autenticación en el cliente

- `lib/stores/auth-store.ts` (Zustand): guarda `user` y `accessToken` solo en memoria (nunca en `localStorage`) — si se recarga la página, se pierden, y se recuperan llamando a `/auth/refresh` (que sí tiene la cookie httpOnly persistente). Esto es intencional: guardar el access token en `localStorage` lo expondría a cualquier script que corra en la página (XSS), mientras que en memoria de JS solo es accesible mientras la pestaña vive.
- `components/auth-guard.tsx` : al montar, si no hay sesión en memoria, intenta silenciosamente `/auth/refresh` (usando la cookie) antes de decidir mandar a `/login` — así una recarga de página (F5) no desloguea al usuario mientras su refresh token siga vigente.
- `components/require-admin.tsx` : guard adicional para `/usuarios` — si el usuario autenticado no es `ADMIN` , redirige a `/dashboard` con `router.replace` (no dependas de esto como seguridad real: el backend ya rechaza esas requests con 403 de todos modos, este componente solo evita que un vendedor vea la UI sin sentido).
- `lib/api.ts` : instancia de `axios` con dos interceptores:
 - *Request*: adjunta `Authorization: Bearer <token>` leyendo el store.
 - *Response*: si una request falla con 401 (access token expirado), llama una vez a `/auth/refresh` en un cliente axios separado (para no reentrar en el mismo interceptor), guarda el nuevo access token, y **reintenta automáticamente** la request original. Si el refresh también falla, limpia la sesión y redirige a `/login` . Este patrón hace que el resto de la app nunca tenga que pensar en "¿expiró el token?" — simplemente falla y se recupera de forma transparente.

6.3 Estado de servidor: TanStack Query + hooks por dominio

Cada dominio (`products` , `clients` , `invoices` , `categories` , `users` , `dashboard`) tiene un archivo `lib/hooks/use-<dominio>.ts` con hooks como `useProducts(filters)` , `useCreateProduct()` , `useUpdateProduct()` , etc.

Cada uno envuelve `useQuery` / `useMutation` de TanStack Query. Patrón común a todos: la API del backend devuelve sobres de respuesta —

`{data, page, pageSize, total}` para listados, `{product: {...}}` para una mutación de un producto individual, `{invoice: {...}}` , `{user: {...}}` , etc. — y cada `queryFn` / `mutationFn` desenvuelve ese sobre antes de devolver el dato al componente, para que el resto del frontend trabaje directamente con el objeto de dominio, no con la envoltura HTTP.

Las mutaciones (crear/editar/borrar) invalidan las queries relacionadas al terminar (`queryClient.invalidateQueries(...)`), lo cual dispara un refetch automático de cualquier lista que esté visible en pantalla — así, por ejemplo, crear un producto nuevo actualiza la tabla de productos sin recargar la página manualmente.

`Providers` (`components/providers.tsx`) configura el `QueryClient` global con `staleTime: 30s` y `refetchOnWindowFocus: false` (evita refetches agresivos cada vez que el usuario vuelve a la pestaña), y envuelve todo en `ThemeProvider` (modo claro/oscurito vía `next-themes`) y `<Toaster>` (los mensajes de éxito/error, con `sonner`).

6.4 Formularios: React Hook Form + Zod

Cada formulario de dominio (`lib/schemas/*.ts`) define su esquema Zod (las mismas reglas de validación del backend, duplicadas intencionalmente en el frontend para dar feedback instantáneo sin esperar un roundtrip HTTP — el backend igual vuelve a validar todo, porque nunca hay que confiar en la validación del cliente). `react-hook-form` con `zodResolver` conecta ese esquema a los inputs y maneja estados de error/touched/dirty por campo.

6.5 Componentes UI base (`components/ui/`)

Basados en el estilo shadcn "base-nova", contruidos sobre `@base-ui/react` (no Radix — una distinción importante porque las APIs y comportamientos por defecto difieren de lo que la documentación general de shadcn describe).

Algunas particularidades de Base UI descubiertas y resueltas durante el desarrollo:

- Un `Button` que renderiza como link (`render={<Link href="..." />}`) necesita `nativeButton={false}` explícito, o Base UI emite una advertencia en consola sobre semántica de botón nativo.
- `Select` se vuelve "no controlado → controlado" (advertencia de React) si su `value` inicial es `undefined` — por eso varios formularios usan un valor centinela explícito (por ejemplo `"NONE"` para "sin categoría") en vez de dejar el campo sin valor inicial.
- `SelectValue` por defecto muestra el `value` crudo (por ejemplo, un uuid), no una etiqueta legible — hace falta pasarle un `children` como render-prop que mapee `value → label` explícitamente.
- `Select.Root` renderiza, oculto, un `<input>` nativo `position: fixed` como hijo — ese elemento fuera de flujo normal seguía recibiendo `margin-top` de utilidades `space-y-*` de Tailwind (que aplican margen al hermano siguiente en el DOM, sin importar si está fuera de flujo), desplazando visualmente el trigger visible unos pixeles hacia abajo respecto a un `<Input>` normal en el mismo formulario. La solución fue usar `flex flex-col gap-` en vez de `space-y-` en esos contenedores específicos — `gap` no afecta a elementos fuera de flujo.
- `Dialog` / `AlertDialog` no traían sombra por defecto, a diferencia de `Select` / `Popover` / `DropdownMenu` / `Sheet` — se igualó agregando `shadow-2xl` a su contenido.

6.6 `components/layout/app-shell.tsx`

El "cascarón" de toda la sección protegida: sidebar fijo a la izquierda + header + área de contenido scrolleable a la derecha. Los ítems de

navegación (`NAV_ITEMS`) tienen un flag `adminOnly: boolean` ; la lista visible se filtra en tiempo de render (`visibleNavItems = NAV_ITEMS.filter( item => !item.adminOnly || isAdmin)`), así que un vendedor simplemente nunca ve el ítem "Usuarios" en el menú — aunque, como se explicó antes, esto es solo UX: la protección real está en `requireRole("ADMIN")` del backend y en `RequireAdmin` como defensa adicional en el frontend.

El bug de scroll corregido: el wrapper raíz originalmente usaba `min-h-screen` sin controlar `overflow` , en un layout `flex` de fila (sidebar + contenido). Con `min-h-screen` , si el contenido de una página era más alto que la pantalla, **toda la página** (incluyendo el sidebar) scrolleaba junta, así que el sidebar "se iba" con el scroll en vez de quedar fijo. La corrección fue `h-screen overflow-hidden` tanto en el wrapper raíz como en la columna derecha, dejando que únicamente el `<main>` tenga su propio `overflow-y-auto` — el patrón estándar de "shell fijo + contenido scrolleable independiente".

6.7 Páginas y componentes de dominio — recorrido rápido

- `(auth)/login/page.tsx` : formulario de login (`LoginForm`) dentro del layout split-screen (panel izquierdo con branding, visible solo en pantallas grandes; panel derecho con el formulario, siempre visible).
- `dashboard/page.tsx` : KPIs (`KpiCard`) + gráfico de ventas (`SalesChart` , un bar chart de un solo color por diseño — revisado contra las reglas de visualización de datos del proyecto: hue único, extremos redondeados, líneas de cuadrícula horizontales discretas, tooltip personalizado, nunca doble eje) + tabla de productos más vendidos.
- `productos/page.tsx` : tabla paginada + `ProductFormDialog` (crear/editar, incluye selección/subida de imagen) + `StockMovementDialog` (ajustar stock) + `CategoryManagerDialog` .
- `clientes/page.tsx` : tabla + `ClientFormDialog` .
- `facturas/page.tsx` : listado con filtro de estado (Select) y botón "Nueva factura" que navega a `facturas/nueva` .
- `facturas/nueva/page.tsx` : selección de cliente (`ClientCombobox`),

agregado de líneas de producto (`ProductAutocomplete` + `InvoiceLineItems`), cálculo de subtotal/impuesto/total en vivo (`lib/invoice-calculations.ts`) antes de enviar al backend (que vuelve a calcular todo del lado servidor — el cálculo en el cliente es solo para mostrarle el total al usuario antes de confirmar).

- `facturas/[id]/page.tsx` : detalle de una factura — datos de cliente, tabla de líneas, totales, badge de estado, botón de anular (solo visible para admin y solo si la factura sigue `EMITIDA` , con confirmación vía `ConfirmDialog`) y botón de descargar PDF. Si la factura está `ANULADA` , muestra quién la anuló y cuándo (los campos `cancelledBy` / `cancelledAt` que vienen del backend).

- `usuarios/page.tsx` : envuelta en `RequireAdmin` , tabla de usuarios (`UserTable` , con badges de rol y estado activo) + `UserFormDialog` (crear/editar — email deshabilitado en modo edición, contraseña solo pedida/requerida en modo creación).

`components/confirm-dialog.tsx` : un `ConfirmDialog` reutilizable (envuelve `AlertDialog` de shadcn) que reemplaza a `window.confirm()` en **todas** las acciones destructivas o irreversibles de la app (anular factura, borrar producto, etc.) — `window.confirm()` es un diálogo nativo del navegador, bloqueante, que además no se puede estilizar ni testear bien con automatización de navegador.

7. Flujos completos, paso a paso

7.1 Login y sesión persistente entre recargas

1. Usuario llena el formulario en `/login` → `LoginForm` llama a `POST /auth/login` con `{email, password}` .
1. Backend valida credenciales, crea un refresh token, y responde `{user, accessToken}` **en el body**, mientras que el refresh token viaja como cookie `httpOnly` en la respuesta HTTP (el navegador la guarda automáticamente, JS nunca la toca).

1. El frontend guarda `user` / `accessToken` en el store de Zustand (memoria) y redirige a `/dashboard`.

1. Cada request subsiguiente lleva `Authorization: Bearer <accessToken>` (interceptor de axios).

1. Si el usuario recarga la página (F5), la memoria de Zustand se pierde, pero `AuthGuard` detecta que no hay sesión y llama a ``POST /auth/refresh`` — el navegador manda la cookie automáticamente, el backend valida el refresh token, rota (revoca el viejo, crea uno nuevo) y devuelve un access token fresco. `AuthGuard` repuebla el store y la sesión sigue como si nada.

1. A los 15 minutos, cualquier request con el access token viejo devuelve 401 → el interceptor de respuesta de axios dispara el mismo flujo de refresh automáticamente, y reintenta la request original — invisible para el usuario.

1. Si el refresh también falla (el refresh token expiró a los 7 días, o fue revocado), se limpia la sesión y se redirige a `/login`.

7.2 Crear una factura

1. En `/facturas/nueva`, el usuario elige un cliente (`ClientCombobox`, busca clientes vía `useClients` con debounce) y agrega líneas de producto (`ProductAutocomplete` busca por nombre/SKU).

1. El frontend calcula subtotal/impuesto/total en vivo solo para mostrarlos (`lib/invoice-calculations.ts`) — es una previsualización, no la fuente de verdad.

1. Al confirmar, `useCreateInvoice().mutateAsync({clientId, items})` llama a `POST /invoices`.

1. El backend, dentro de una transacción: valida stock de cada línea, calcula los montos reales server-side con `Prisma.Decimal`, crea `Invoice` + `InvoiceItem[]`, descuenta `Product.stock` y registra `StockMovement` tipo `SALIDA` por cada línea.

1. Si algún producto no tiene stock suficiente, la transacción completa se aborta y el backend responde 409 con el detalle exacto de qué producto y

cuánto faltaba — el frontend muestra ese mensaje vía

`getApiErrorMessage()`, sin haber tocado el stock para nada.

1. Si todo sale bien, responde 201 con `{invoice: {...}}`, y el frontend redirige al detalle de la factura recién creada.

7.3 Subir la imagen de un producto

1. En `ProductFormDialog`, si es modo **creación**, `ProductImagePicker` deja elegir un archivo local (genera solo una vista previa con `URL.createObjectURL`, vía `useMemo`, sin subir nada todavía). Si es modo **edición**, `ProductImageField` sube/cambia/borra directamente contra la API en cuanto el usuario actúa.

1. En creación: al enviar el formulario, primero se crea el producto (`POST /products`), y **solo si eso tiene éxito**, se sube la imagen elegida (`POST /products/:id/image`, multipart) en una llamada separada con su propio try/catch — así, si la subida de imagen fallara por cualquier motivo, el producto ya creado no se pierde ni se marca como error; solo se le avisa al usuario que la imagen no se pudo subir.

1. El backend valida tamaño/tipo declarado (multer) → valida la firma real de los bytes (`assertValidImageContent`) → sube a R2 → borra la imagen anterior de R2 si existía → guarda la nueva URL en `Product.imageUrl`.

1. El frontend invalida la query de ese producto, así que la tabla/el detalle reflejan la nueva imagen sin recargar la página.

7.4 Anular una factura

1. Solo un admin ve el botón "Anular factura" en `/facturas/[id]`, y solo si `status === "EMITIDA"`.

1. Al hacer clic, se abre `ConfirmDialog` (nunca `window.confirm`) pidiendo confirmación explícita, con una advertencia de que la acción repone stock y no se puede deshacer.

1. Al confirmar, `useCancelInvoice().mutateAsync(invoice.id)` llama a `PATCH /invoices/:id/cancel`.

1. Backend (dentro de transacción): repone el stock de cada línea, registra

un `StockMovement` tipo `ENTRADA` a nombre de **quien anula** (el admin actual, tomado de `req.user.sub`, no del `userId` original de la factura), y marca la factura `ANULADA` con `cancelledAt` / `cancelledByUserId`.

1. El frontend refleja el nuevo estado inmediatamente (badge "Anulada" + la línea "Anulada por <nombre> el <fecha>").

8. Seguridad — resumen de decisiones concretas

- Contraseñas con `bcrypt` costo 12; nunca se loguean ni se devuelven en ninguna respuesta HTTP.
- JWT de acceso de vida corta (15 min) + refresh token opaco, hasheado, rotado en cada uso, en cookie `httpOnly/secure/sameSite=strict`.
- Toda mutación pasa por validación Zod antes de tocar la base de datos; todo listado paginado tiene un `pageSize` máximo (100) para evitar respuestas gigantes.
- `requireAuth` + `requireRole` en cada ruta que lo necesita — la autorización se revalida en el backend siempre, independientemente de lo que el frontend oculte.
- Rate limiting en dos capas: estricto en login (5/15min), general en toda la API (600/15min) — con la limitación conocida de ser en memoria del proceso (no comparte estado entre instancias).
- Imágenes: validación de tamaño + tipo declarado + **contenido real** (firma binaria) antes de subir a almacenamiento externo.
- El seed se niega a correr en producción con la contraseña de admin por defecto.
- Nunca se expone `passwordHash` ni `tokenHash` — ni por descuido: además del `select` explícito en cada query, `serialize()` los filtra como red de seguridad final.
- Cada acción de anulación de factura queda auditada con quién y cuándo.

9. Cómo correr el proyecto localmente

```
# Base de datos (Postgres vía Docker)
docker compose up -d

# Backend
cd backend
cp .env.example .env          # completar JWT_ACCESS_SECRET, REFRESH_TOKEN_SECRET, etc.
npm install
npx prisma migrate deploy    # aplica las migraciones
npx prisma db seed           # crea el admin + categorías de ejemplo
npm run dev                  # levanta en el puerto configurado (PORT)

# Frontend (otra terminal)
cd frontend
cp .env.example .env.local    # apuntar NEXT_PUBLIC_API_URL al backend
npm install
npm run dev                   # levanta en localhost:3000
```

Tests:

```
cd backend && npm test      # Vitest + supertest, contra .env.test
cd frontend && npm test     # tests de componentes/hooks
```

10. Índice rápido — "¿dónde toco esto?"

Quiero...	Archivo(s)
Cambiar el % de impuesto	<code>backend/.env</code> → <code>TAX_RATE</code>
Agregar un campo a Producto	<code>prisma/schema.prisma</code> (+ migración) → <code>products.schemas.ts</code> → <code>products.service.ts</code> → <code>frontend/lib/types.ts</code> → <code>ProductFormDialog</code>
Cambiar reglas de quién ve qué	<code>requireRole(...)</code> en cada <code>*.routes.ts</code> , y <code>NAV_ITEMS</code> / <code>adminOnly</code> en <code>app-shell.tsx</code>
Cambiar duración de sesión	<code>ACCESS_TOKEN_TTL</code> / <code>REFRESH_TOKEN_TTL_MS</code> en <code>auth.service.ts</code>
Cambiar el límite de rate limiting	<code>middlewares/rateLimit.ts</code>
Cambiar el diseño de un formulario	el <code>*-form-dialog.tsx</code> correspondiente en <code>components/<dominio>/</code>

Quiero...	Archivo(s)
Ver el contrato exacto de un endpoint	<code>docs/api-contract.md</code>
Ver el historial de bugs encontrados y corregidos	<code>docs/04-qa-gate.md</code>

Este manual refleja el estado del código al 2026-09-17. Si el código cambia, este documento puede quedar desactualizado — ante la duda, el código siempre es la fuente de verdad; este manual es un mapa, no el territorio.